Stack, Queue & Priority Queue Templates

Four data structures — each with a canonical template and representative problems.

Quick Reference Table

#	Name	Structure	Pattern	Pop/poll condition	Return
20	Valid Parentheses	Stack	Bracket matching	pop on close bracket	boolean
155	Min Stack	Stack + aux stack	Mirror min alongside values	always pop both	int (min)
394	Decode String	Two stacks	Push on `[`, expand on `]`	pop on `]`	String
739	Daily Temperatures	Mono dec stack	Next greater temp	curr > top	int[]
496	Next Greater Element I	Mono dec stack + Map	NGE across two arrays	curr > top	int[]
84	Largest Rectangle	Mono inc stack	Next smaller bar → compute width	curr < top	int (area)
42	Trapping Rain Water	Mono inc stack	Valley between bars	curr > top	int (water)
239	Sliding Window Max	Mono dec deque	Window max at front	pollLast when curr ≥ back	int[]
1046	Last Stone Weight	Max-heap	Greedily smash top 2	always poll 2	int
347	Top K Frequent	Min-heap size k	Evict smallest when size > k	size > k → poll	int[]
295	Find Median	Max-heap + Min-heap	Lower/upper halves	rebalance after each add	double
502	IPO	Sort + Max-heap	Unlock by capital, pick max profit	w >= required capital	int
767	Reorganize String	Max-heap	Interleave two most frequent chars	always poll 2	String
85	Maximal Rectangle	Largest rectangle containing only 1s in binary matrix	For each row build histogram heights → apply #84 largestRectangleArea	**Row-by-row histogram**: reduce 2D problem to repeated #84	O(m·n)
224	Basic Calculator	Evaluate expression with +, -, ( )	Stack saves (result, sign) before each `(`; restore after `)`	**Sign + parentheses stack**: push (result,sign) on `(`, pop and combine on `)`	O(n)
227	Basic Calculator II	Evaluate expression with +, -, *, / (no parentheses)	Scan tokens; push negative/positive for +/-; multiply/divide top of stack for */÷	**Operator-before-operand**: apply *, / immediately; defer +, - via signed push	O(n)
373	Find K Pairs with Smallest Sums	Find k pairs (u,v) with smallest u+v, u from nums1, v from nums2	Min-heap seeded with (nums1[i], nums2[0]) for all i; expand j on pop	**Expand j**: each heap entry tracks (i, j); pop → push (i, j+1)	O(k log k)
378	Kth Smallest Element in Sorted Matrix	kth smallest in n×n matrix where rows and cols are sorted	Binary search on value range [min,max]; count elements ≤ mid from bottom-left	**Binary search on value**: count(mid) ≥ k → search lower; else search higher	O(n log(max-min))
503	Next Greater Element II	Next greater element in circular array	Monotone decreasing stack; iterate array twice (i % n)	**Circular**: loop 2n, use i % n; only push indices in first pass (i < n)	O(n)
621	Task Scheduler	Minimum intervals to finish tasks with cooldown n	Count frequencies; answer = max((maxFreq-1)*(n+1)+maxCount, tasks.length)	**Greedy formula**: no simulation needed, direct calculation	O(k) k=tasks.length
358	Rearrange String k Distance Apart	Rearrange so same chars are ≥k apart	Max-heap by frequency + cooldown queue of size k	**Cooldown queue**: hold used char until k slots pass, then re-add to heap	O(n log k)
480	Sliding Window Median	Median of each window of size k	Two heaps (maxHeap lower, minHeap upper) + lazy deletion via HashMap	**Lazy deletion**: defer removing out-of-window elements; rebalance by counts	O(n log k)
692	Top K Frequent Words	k most frequent words, lexicographic on ties	Min-heap of size k ordered by (freq asc, word desc)	**Custom tie-break**: same freq → larger word first so it's evicted	O(n log k)

#	Name	Structure	Pattern	Pop/poll condition	Return
772	Basic Calculator III	Evaluate expression with +,-,*,/ and parentheses	Recursive/stack: handle parentheses recursively, apply *,/ immediately	**Full calculator**: combine #224 parentheses + #227 operator precedence	O(n)

All Templates

```
// 1. STACK (LIFO) — use ArrayDeque, not Stack class
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);     // add to top
stack.pop();       // remove from top
stack.peek();      // view top, no remove

// 2A. MONOTONE DECREASING STACK — next GREATER element
// keep only candidates still waiting for a bigger neighbor; current resolves them all.  — WHEN: "next/
// Invariant: stack values decrease from bottom to top
// Pop when current is GREATER than top → top found its next greater
Deque<Integer> stack = new ArrayDeque<>();  // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i])
        result[stack.pop()] = nums[i];     // nums[i] is next greater for popped index
    stack.push(i);
}
// Remaining in stack: no next greater element exists

// 2B. MONOTONE INCREASING STACK — next SMALLER element / span width
// each popped bar's reach extends until something shorter blocks it on both sides.  — WHEN: "largest r
// Invariant: stack values increase from bottom to top
// Pop when current is SMALLER than top → use popped index to compute width/span
Deque<Integer> stack = new ArrayDeque<>();  // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
        int height = nums[stack.pop()];
        int width  = stack.isEmpty() ? i : i - stack.peek() - 1;  // span between boundaries
        // process height * width
    }
    stack.push(i);
}

// 3. MONOTONE DEQUE — sliding window max/min
// the front is the current window's answer; anything a newer-and-bigger value beats is dead weight.  —
// Deque stores INDICES; values at those indices are decreasing front-to-back (for max)
Deque<Integer> deque = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i])
        deque.pollLast();              // remove smaller elements — they can never be window max
    deque.offerLast(i);
    if (deque.peekFirst() < i - k + 1) deque.pollFirst();  // evict out-of-window elements
    if (i >= k - 1) result[i - k + 1] = nums[deque.peekFirst()];
}

// 4. PRIORITY QUEUE
PriorityQueue<Integer> minPQ = new PriorityQueue<>();                     // min-heap (default)
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
PriorityQueue<int[]>    pq   = new PriorityQueue<>((a, b) -> a[1] - b[1]); // custom: sort by index 1
pq.offer(x);   // add
pq.poll();     // remove and return min/max
pq.peek();     // view min/max without removing

// 5. TWO HEAPS — maintain median in O(log n)
// split the data at the median; the median is always sitting on the two heaps' tops.  — WHEN: "running
// maxHeap = lower half (smaller numbers), minHeap = upper half (larger numbers)
// Invariant: maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size() + 1
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
void addNum(int num) {
    maxHeap.offer(num);
```

```
    minHeap.offer(maxHeap.poll());              // push one to minHeap (possibly num itself)
    if (maxHeap.size() < minHeap.size())
        maxHeap.offer(minHeap.poll());          // rebalance: maxHeap always ≥ minHeap in size
}
double findMedian() {
    return maxHeap.size() > minHeap.size()
        ? maxHeap.peek()
        : (maxHeap.peek() + minHeap.peek()) / 2.0;
}
```

Part 1 — Basic Stack

#20 Valid Parentheses

Description: Given a string of brackets `()[]{}`, return true if all brackets are properly closed and ordered.

Intuition: the most recent unmatched open bracket is the only one a closing bracket can legally match — that is exactly LIFO.

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') stack.push(c);
            else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if (c == ')' && top != '(') return false;
                if (c == ']' && top != '[') return false;
                if (c == '}' && top != '{') return false;
            }
        }
        return stack.isEmpty();
    }
}
```

Time O(n) | **Space** O(n)

#155 Min Stack

Description: Design a stack that supports push, pop, top, and `getMin()` in O(1).
Key: auxiliary `minStack` mirrors the main stack — each level stores the current running minimum.

```
class MinStack {
    Deque<Integer> stack    = new ArrayDeque<>();
    Deque<Integer> minStack = new ArrayDeque<>();   // ← always push current min alongside value

    public void push(int val) {
        stack.push(val);
        minStack.push(minStack.isEmpty() ? val : Math.min(minStack.peek(), val));
    }
    public void pop()    { stack.pop(); minStack.pop(); }  // ← pop both in sync
    public int top()     { return stack.peek(); }
    public int getMin()  { return minStack.peek(); }
}
```

Time O(1) per operation | **Space** O(n)

#394 Decode String

Description: Decode a string like `"3[a2[bc]]"` → `"abcbcabcbc"`.
Key: two stacks — one for repeat counts, one for the string built before each `[`. On `]`, pop both and repeat.

```
class Solution {
    public String decodeString(String s) {
        Deque<Integer>      countStack = new ArrayDeque<>();
        Deque<StringBuilder> strStack  = new ArrayDeque<>();
        StringBuilder current = new StringBuilder();
        int k = 0;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                k = k * 10 + (c - '0');                          // ← multi-digit number
            } else if (c == '[') {
                countStack.push(k);                               // ← save count
                strStack.push(current);                           // ← save string so far
                current = new StringBuilder();
                k = 0;
            } else if (c == ']') {
                int count = countStack.pop();
                StringBuilder parent = strStack.pop();
                for (int i = 0; i < count; i++) parent.append(current);  // ← repeat
                current = parent;
            } else {
                current.append(c);
            }
        }
        return current.toString();
    }
}
```

Time O(n) (n = length of decoded output) | **Space** O(n)

Part 2 — Monotone Stack

Decreasing vs Increasing

```
DECREASING stack (pop when current > top):     finds NEXT GREATER element for each popped index
INCREASING stack (pop when current < top):     finds NEXT SMALLER element — used for span width

Both store INDICES (not values) so you can compute distances and widths.
```

#739 Daily Temperatures

Description: For each day, how many days until a warmer temperature? Return 0 if none.
Template: monotone decreasing — when current temp is greater, pop and record the gap.

Intuition: a colder day just waits on the stack until the first warmer day arrives and resolves it.

```java
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] result = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();   // indices; temps decrease from bottom to top
        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && temperatures[stack.peek()] < temperatures[i]) {
                int idx = stack.pop();
                result[idx] = i - idx;               // ← days until warmer = distance
            }
            stack.push(i);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#496 Next Greater Element I

Description: For each element in `nums1` (a subset of `nums2`), find the first greater element to its right in `nums2`. Return -1 if none.
Key: precompute NGE for all elements in `nums2` using a monotone stack + HashMap. Then look up each `nums1` element.

```java
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        Map<Integer, Integer> nextGreater = new HashMap<>();
        Deque<Integer> stack = new ArrayDeque<>();
        for (int num : nums2) {
            while (!stack.isEmpty() && stack.peek() < num)
                nextGreater.put(stack.pop(), num);   // ← num is next greater for stack.peek()
            stack.push(num);
        }
        int[] result = new int[nums1.length];
        for (int i = 0; i < nums1.length; i++)
            result[i] = nextGreater.getOrDefault(nums1[i], -1);
        return result;
    }
}
```

Time O(m + n) | **Space** O(n)

#84 Largest Rectangle in Histogram

Description: Find the largest rectangle that can be formed within the histogram bars.
Template: monotone increasing stack (pop when current bar is shorter). Popped bar's width = from `stack.peek() + 1` to `i - 1`.
Key: add sentinel `0`s at both ends to flush all remaining bars from the stack.

Intuition: a bar can only stretch sideways until it hits a shorter bar; the stack remembers each bar's left limit so the right limit (current shorter bar) closes the rectangle.

```java
class Solution {
    public int largestRectangleArea(int[] heights) {
        int n = heights.length;
        int[] h = new int[n + 2];                          // ← sentinels: h[0]=h[n+1]=0
        System.arraycopy(heights, 0, h, 1, n);
        Deque<Integer> stack = new ArrayDeque<>();
        int maxArea = 0;
        for (int i = 0; i <= n + 1; i++) {
            while (!stack.isEmpty() && h[stack.peek()] > h[i]) {
                int height = h[stack.pop()];
                int left   = stack.isEmpty() ? -1 : stack.peek();  // left boundary
                int width  = i - left - 1;                 // bars strictly between left boundary and cu
                maxArea = Math.max(maxArea, height * width);
            }
            stack.push(i);
        }
        return maxArea;
    }
}
```

Time O(n) | **Space** O(n)

#42 Trapping Rain Water

Description: How much water can be trapped between the bars after rain?
Template: monotone increasing stack — when current bar is taller, water fills the "valley" between the current bar and the bar now at the top of the stack.

Intuition: water sits in a dip between a left wall and a right wall, and its depth is set by whichever wall is shorter.

```java
class Solution {
    public int trap(int[] height) {
        Deque<Integer> stack = new ArrayDeque<>();
        int water = 0;
        for (int i = 0; i < height.length; i++) {
            while (!stack.isEmpty() && height[stack.peek()] < height[i]) {
                int bottom   = height[stack.pop()];
                if (stack.isEmpty()) break;
                int left     = stack.peek();
                int h        = Math.min(height[left], height[i]) - bottom;  // bounded by the shorter w
                int w        = i - left - 1;
                water       += h * w;
            }
            stack.push(i);
        }
        return water;
    }
}
```

Time O(n) | **Space** O(n)

84 vs 42 — Side by Side

```
                  #84 Largest Rectangle       #42 Trapping Rain Water
Pop when:         h[curr] < h[top]            h[curr] > h[top]
Stack order:      increasing (bottom→top)      increasing (bottom→top)
Popped bar role:  height of rectangle          floor of the trapped water valley
Width from:       (i - stack.peek() - 1)       (i - stack.peek() - 1)
Area/water:       height × width               min(left_wall, right_wall) × width
```

Part 3 — Monotone Deque

#239 Sliding Window Maximum

Description: Return the maximum in each sliding window of size k.
Template: deque stores indices; values at those indices are strictly decreasing front-to-back (max at front).

Intuition: any element smaller than a newer element can never be the window max again, so discard it; the front always holds the current best.

```java
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque<Integer> deque = new ArrayDeque<>();   // indices, values decrease front→back
        for (int i = 0; i < n; i++) {
            while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i])
                deque.pollLast();                    // ← remove smaller: they can never be max
            deque.offerLast(i);
            if (deque.peekFirst() < i - k + 1)
                deque.pollFirst();                   // ← evict index out of window
            if (i >= k - 1)
                result[i - k + 1] = nums[deque.peekFirst()];  // ← front = window max
        }
        return result;
    }
}
```

Time O(n) | **Space** O(k)

Part 4 — Priority Queue

#1046 Last Stone Weight

Description: Smash the two heaviest stones repeatedly. Return the last remaining weight (or 0).
Template: max-heap — always poll the two largest.

```java
class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
        for (int stone : stones) pq.offer(stone);
        while (pq.size() > 1) {
            int a = pq.poll(), b = pq.poll();
            if (a != b) pq.offer(a - b);            // ← only add remainder if different
        }
        return pq.isEmpty() ? 0 : pq.poll();
    }
}
```

Time O(n log n) | **Space** O(n)

#347 Top K Frequent Elements

Description: Return the k most frequent elements.
Template: min-heap of size k — evict the least frequent when size exceeds k. What remains = top k.

```java
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int num : nums) freq.merge(num, 1, Integer::sum);
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);  // min-heap on freq
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            pq.offer(new int[]{e.getKey(), e.getValue()});
            if (pq.size() > k) pq.poll();           // ← evict least frequent
        }
        return pq.stream().mapToInt(a -> a[0]).toArray();
    }
}
```

Time O(n log k) | **Space** O(n)

#295 Find Median from Data Stream

Description: Add integers to a stream and find the median at any time in O(log n) add and O(1) find.
Template: two heaps — `maxHeap` holds lower half, `minHeap` holds upper half. Always rebalance so `maxHeap.size() ≥ minHeap.size()`.

Intuition: keep the smaller half and larger half balanced; the median always lives right at the boundary, on the tops of the two heaps.

```
class MedianFinder {
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // lower half
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();                           // upper half

    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());               // ← always push one to minHeap first
        if (maxHeap.size() < minHeap.size())
            maxHeap.offer(minHeap.poll());           // ← rebalance: maxHeap always ≥ minHeap size
    }
    public double findMedian() {
        return maxHeap.size() > minHeap.size()
            ? maxHeap.peek()
            : (maxHeap.peek() + minHeap.peek()) / 2.0;
    }
}
```

Time O(log n) addNum, O(1) findMedian | **Space** O(n)

#502 IPO

Description: Before each project you must have enough capital. Start with capital `w`. Pick at most `k` projects to maximize final capital.
Template: sort by required capital + max-heap of profits. At each step, unlock all affordable projects (move them into the heap), then greedily pick the most profitable.

```
class Solution {
    public int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
        int n = profits.length;
        int[][] projects = new int[n][2];
        for (int i = 0; i < n; i++) projects[i] = new int[]{capital[i], profits[i]};
        Arrays.sort(projects, (a, b) -> a[0] - b[0]);              // ← sort by required capital
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder()); // max-heap of pro
        int idx = 0;
        for (int i = 0; i < k; i++) {
            while (idx < n && projects[idx][0] <= w) pq.offer(projects[idx++][1]); // ← unlock
            if (pq.isEmpty()) break;
            w += pq.poll();                                         // ← pick most profitable
        }
        return w;
    }
}
```

Time O(n log n) | **Space** O(n)

#767 Reorganize String

Description: Rearrange characters so no two adjacent characters are the same. Return `""` if impossible.
Template: max-heap of (char, count) — at each step, pick the two most frequent chars and append them alternately.

```java
class Solution {
    public String reorganizeString(String s) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) count[c - 'a']++;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[1] - a[1]); // max-heap on count
        for (int i = 0; i < 26; i++) if (count[i] > 0) pq.offer(new int[]{i, count[i]});
        StringBuilder sb = new StringBuilder();
        while (pq.size() > 1) {
            int[] a = pq.poll(), b = pq.poll();
            sb.append((char)('a' + a[0]));
            sb.append((char)('a' + b[0]));
            if (--a[1] > 0) pq.offer(a);
            if (--b[1] > 0) pq.offer(b);
        }
        if (!pq.isEmpty()) {
            int[] last = pq.poll();
            if (last[1] > 1) return "";                          // ← only one char left, count > 1:
            sb.append((char)('a' + last[0]));
        }
        return sb.toString();
    }
}
```

Time O(n log k) (k = distinct chars ≤ 26) | **Space** O(k)

Structure Chooser

Goal	Structure	Why
Next greater/smaller element	Monotone stack	O(n) total — each element pushed/popped once
Sliding window max/min	Monotone deque	Front = window extreme; stale indices evicted
Global min/max, k-th element	PriorityQueue	O(log n) per op
Running median	Two heaps	Split at median; each heap half is O(log n)
O(1) stack minimum	Aux min-stack	Mirror min value at every level
Nested structure (brackets, k-repeats)	Two stacks	One for counts, one for strings

Deque API Cheat Sheet

```java
Deque<Integer> d = new ArrayDeque<>();

// As STACK (LIFO):    push()/pop()/peek()  → operate on FRONT
d.push(x);    // = offerFirst
d.pop();      // = pollFirst
d.peek();     // = peekFirst

// As QUEUE (FIFO):   offer()/poll()/peek() → back-in, front-out
d.offerLast(x);
d.pollFirst();
d.peekFirst();

// As DEQUE:          explicit first/last
d.offerFirst(x);  d.pollFirst();  d.peekFirst();
d.offerLast(x);   d.pollLast();   d.peekLast();
```

#503 Next Greater Element II

Description: Given a circular integer array, return the next greater number for every element. The next greater number of an element is the first greater number found by traversing the array circularly.

Variation: Monotone decreasing stack. Iterate through the array TWICE (indices 0 to 2n-1, using `i % n`). Only push index `i` onto the stack during the first pass (`i < n`) to avoid duplicates.

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        Arrays.fill(result, -1);
        Deque<Integer> stack = new ArrayDeque<>();   // monotone decreasing stack of indices
        for (int i = 0; i < 2 * n; i++) {           // ← VARIATION: iterate twice for circular
            while (!stack.isEmpty() && nums[stack.peek()] < nums[i % n])
                result[stack.pop()] = nums[i % n];
            if (i < n) stack.push(i);               // ← VARIATION: only push in first pass
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#85 Maximal Rectangle

Description: Given a binary matrix filled with 0s and 1s, find the largest rectangle containing only 1s and return its area.

Variation: For each row, compute cumulative histogram heights (1s stack vertically). Then apply the Largest Rectangle in Histogram algorithm (#84) on each row's histogram.

```java
class Solution {
    public int maximalRectangle(char[][] matrix) {
        int cols = matrix[0].length;
        int[] heights = new int[cols];
        int maxArea = 0;
        for (char[] row : matrix) {
            for (int j = 0; j < cols; j++)
                heights[j] = row[j] == '1' ? heights[j] + 1 : 0;  // ← VARIATION: build histogram row b
            maxArea = Math.max(maxArea, largestRectangle(heights));
        }
        return maxArea;
    }

    private int largestRectangle(int[] heights) {
        int n = heights.length;
        // Pad with 0s on both ends to flush remaining stack
        int[] h = new int[n + 2];
        System.arraycopy(heights, 0, h, 1, n);
        Deque<Integer> stack = new ArrayDeque<>();
        int max = 0;
        for (int i = 0; i <= n + 1; i++) {
            while (!stack.isEmpty() && h[stack.peek()] > h[i]) {
                int height = h[stack.pop()];
                max = Math.max(max, height * (i - stack.peek() - 1));
            }
            stack.push(i);
        }
        return max;
    }
}
```

Time O(m·n) | **Space** O(n)

#224 Basic Calculator

Description: Evaluate a string expression containing `+`, `-`, `(`, `)`, digits, and spaces.

Variation: When encountering `(`, push current (result, sign) onto stack and reset. When encountering `)`, combine current result with the saved result and sign from the stack.

```java
class Solution {
    public int calculate(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int result = 0, number = 0, sign = 1;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                number = number * 10 + (c - '0');
            } else if (c == '+') {
                result += sign * number; number = 0; sign = 1;
            } else if (c == '-') {
                result += sign * number; number = 0; sign = -1;
            } else if (c == '(') {
                stack.push(result);    // ← VARIATION: save result and sign before '('
                stack.push(sign);
                result = 0; sign = 1;
            } else if (c == ')') {
                result += sign * number; number = 0;
                result *= stack.pop();  // ← VARIATION: multiply by sign before '('
                result += stack.pop();  // ← VARIATION: add result before '('
            }
        }
        return result + sign * number;
    }
}
```

Time O(n) | **Space** O(n)

#227 Basic Calculator II

Description: Evaluate a string expression with `+`, `-`, `*`, `/` and integers (no parentheses).

Variation: Process operator BEFORE the current number. Push positive/negative numbers for `+`/`-`. For `*`/`/`, immediately multiply/divide the top of the stack. Sum the stack at the end.

```java
class Solution {
    public int calculate(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int number = 0;
        char sign = '+';                                    // ← VARIATION: track previous operator
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (Character.isDigit(c)) number = number * 10 + (c - '0');
            if ((!Character.isDigit(c) && c != ' ') || i == s.length() - 1) {
                if      (sign == '+') stack.push(number);
                else if (sign == '-') stack.push(-number);
                else if (sign == '*') stack.push(stack.pop() * number);  // ← VARIATION: apply * immedi
                else if (sign == '/') stack.push(stack.pop() / number);  // ← VARIATION: apply / immedi
                sign = c; number = 0;
            }
        }
        return stack.stream().mapToInt(Integer::intValue).sum();
    }
}
```

Time O(n) | **Space** O(n)

#378 Kth Smallest Element in a Sorted Matrix

Description: Given an n×n matrix where each row and column is sorted in ascending order, return the kth smallest element.

Variation: Binary search on the VALUE range [matrix[0][0], matrix[n-1][n-1]]. For a given `mid`, count elements ≤ mid by scanning from the bottom-left corner: if `matrix[row][col] <= mid`, all `row+1` elements in this column (0..row) are ≤ mid.

```
class Solution {
    public int kthSmallest(int[][] matrix, int k) {
        int n = matrix.length;
        int i = matrix[0][0], j = matrix[n-1][n-1];        // ← VARIATION: binary search on value range
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (countLE(matrix, mid, n) >= k) {
                j = mid;
            } else {
                i = mid + 1;
            }
        }
        return i;
    }

    private int countLE(int[][] matrix, int target, int n) {
        int count = 0, row = n - 1, col = 0;               // ← VARIATION: scan from bottom-left
        while (row >= 0 && col < n) {
            if (matrix[row][col] <= target) {
                count += row + 1;
                col++;
            } else {
                row--;
            }
        }
        return count;
    }
}
```

Time O(n log(max-min)) | **Space** O(1)

#373 Find K Pairs with Smallest Sums

Description: Given two sorted arrays `nums1` and `nums2`, return the `k` pairs `(u, v)` (one from each) with the smallest sums.

Variation: Seed min-heap with `(nums1[i], nums2[0])` for i = 0..min(k, n1.length)-1. On each pop, if `j+1 < nums2.length`, push `(nums1[i], nums2[j+1])`. This expands row-by-row in the implicit pair matrix.

```java
class Solution {
    public List<List<Integer>> kSmallestPairs(int[] nums1, int[] nums2, int k) {
        List<List<Integer>> result = new ArrayList<>();
        if (nums1.length == 0 || nums2.length == 0) return result;
        // heap stores [i, j] meaning pair (nums1[i], nums2[j])
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) ->
            nums1[a[0]] + nums2[a[1]] - nums1[b[0]] - nums2[b[1]]);
        for (int i = 0; i < Math.min(nums1.length, k); i++)
            pq.offer(new int[]{i, 0});              // ← VARIATION: seed with (i, 0) for each row
        while (!pq.isEmpty() && result.size() < k) {
            int[] curr = pq.poll();
            result.add(Arrays.asList(nums1[curr[0]], nums2[curr[1]]));
            if (curr[1] + 1 < nums2.length)
                pq.offer(new int[]{curr[0], curr[1] + 1});  // ← VARIATION: expand j in same row
        }
        return result;
    }
}
```

Time O(k log k) | **Space** O(k)

#621 Task Scheduler

Description: Given a list of tasks (characters) and a cooldown `n`, return the minimum number of intervals to finish all tasks. Between two same tasks there must be at least `n` intervals.

Variation: Greedy formula. Find the most frequent task (maxFreq). It creates `maxFreq - 1` "frames" of `n + 1` slots, plus `maxCount` (count of tasks with maxFreq) slots. The answer is `max(tasks.length, (maxFreq - 1) * (n + 1) + maxCount)`.

```java
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] count = new int[26];
        for (char c : tasks) count[c - 'A']++;
        Arrays.sort(count);
        int maxFreq = count[25];
        int maxCount = 0;
        for (int c : count) if (c == maxFreq) maxCount++;          // ← VARIATION: count ties at max
        return Math.max(tasks.length, (maxFreq - 1) * (n + 1) + maxCount);  // ← VARIATION: formula
    }
}
```

Time O(k) | **Space** O(1)

#358 Rearrange String k Distance Apart

Description: Rearrange a string so that the same characters are at least `k` distance apart. Return "" if impossible.
Variation: greedy with a max-heap by frequency plus a cooldown queue of size k that holds recently used characters until they're eligible again.

```java
class Solution {
    public String rearrangeString(String s, int k) {
        if (k <= 1) return s;
        Map<Character, Integer> count = new HashMap<>();
        for (char c : s.toCharArray()) count.merge(c, 1, Integer::sum);
        PriorityQueue<Map.Entry<Character,Integer>> maxHeap =
            new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());
        maxHeap.addAll(count.entrySet());
        Queue<Map.Entry<Character,Integer>> cooldown = new ArrayDeque<>();  // ← VARIATION: cooldown qu
        StringBuilder sb = new StringBuilder();
        while (!maxHeap.isEmpty()) {
            var entry = maxHeap.poll();
            sb.append(entry.getKey());
            entry.setValue(entry.getValue() - 1);
            cooldown.offer(entry);
            if (cooldown.size() < k) continue;             // ← VARIATION: wait k slots before reuse
            var ready = cooldown.poll();
            if (ready.getValue() > 0) maxHeap.offer(ready);
        }
        return sb.length() == s.length() ? sb.toString() : "";
    }
}
```

Time O(n log k) | **Space** O(k)

#480 Sliding Window Median

Description: Return the median of every window of size k as it slides across the array. **Variation:** two heaps (maxHeap = lower half, minHeap = upper half) with lazy deletion — defer removing elements that left the window, tracking balance with counts.

```java
class Solution {
    public double[] medianSlidingWindow(int[] nums, int k) {
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> Integer.compare(b, a));
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        double[] result = new double[nums.length - k + 1];
        for (int i = 0; i < nums.length; i++) {
            if (maxHeap.isEmpty() || nums[i] <= maxHeap.peek()) maxHeap.offer(nums[i]);
            else minHeap.offer(nums[i]);
            if (i >= k) {                                  // ← VARIATION: remove element leaving wind
                int out = nums[i - k];
                if (out <= maxHeap.peek()) maxHeap.remove(out);
                else minHeap.remove(out);
            }
            // rebalance
            while (maxHeap.size() > minHeap.size() + 1) minHeap.offer(maxHeap.poll());
            while (minHeap.size() > maxHeap.size()) maxHeap.offer(minHeap.poll());
            if (i >= k - 1)
                result[i - k + 1] = (k % 2 == 1) ? (double) maxHeap.peek()
                    : ((double) maxHeap.peek() + minHeap.peek()) / 2.0;
        }
        return result;
    }
}
```

Time O(n·k) with heap remove (O(n log k) with indexed sets) | **Space** O(k)

#692 Top K Frequent Words

Description: Return the k most frequent words. Sort by frequency descending; ties broken by lexicographic order. **Variation:** min-heap of size k ordered so the "smallest" (lowest freq, or same freq with lexicographically larger word) sits on top for eviction.

```java
class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        Map<String, Integer> count = new HashMap<>();
        for (String w : words) count.merge(w, 1, Integer::sum);
        PriorityQueue<String> heap = new PriorityQueue<>((a, b) ->
            count.get(a).equals(count.get(b)) ? b.compareTo(a)        // ← VARIATION: larger word evict
                                              : count.get(a) - count.get(b));
        for (String w : count.keySet()) {
            heap.offer(w);
            if (heap.size() > k) heap.poll();
        }
        List<String> result = new ArrayList<>();
        while (!heap.isEmpty()) result.add(heap.poll());
        Collections.reverse(result);
        return result;
    }
}
```

Time O(n log k) | **Space** O(n)

#772 Basic Calculator III

Description: Evaluate an arithmetic expression with `+`, `-`, `*`, `/`, and parentheses. **Variation:** combines #224 (parentheses via recursion) and #227 (operator precedence: apply `*` / `/` immediately, defer `+` / `-`).

```java
class Solution {
    private int i;
    public int calculate(String s) { i = 0; return eval(s); }
    private int eval(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int num = 0;
        char op = '+';
        while (i < s.length()) {
            char c = s.charAt(i++);
            if (Character.isDigit(c)) num = num * 10 + (c - '0');
            if (c == '(') num = eval(s);                    // ← VARIATION: recurse on '('
            if ((!Character.isDigit(c) && c != ' ') || i == s.length()) {
                if      (op == '+') stack.push(num);
                else if (op == '-') stack.push(-num);
                else if (op == '*') stack.push(stack.pop() * num);  // ← VARIATION: precedence
                else if (op == '/') stack.push(stack.pop() / num);
                op = c; num = 0;
            }
            if (c == ')') break;                            // ← VARIATION: return to caller on ')'
        }
        int sum = 0;
        while (!stack.isEmpty()) sum += stack.pop();
        return sum;
    }
}
```

Time O(n) | **Space** O(n)